



KRESZ alapú tanulás segítő játék fejlesztése Unity keretrendszerben

Készítette

Fodor Győző Benedek

Programtervező Informatikus Bsc.

Témavezető

Troll Ede Mátyás

Tanársegéd

EGER, 2026

Tartalomjegyzék

Bevezetés	4
1. Technológiai áttekintés	6
1.1. Unreal Engine	7
1.2. Godot	7
1.3. Unity	8
1.3.1. Komponensek	10
2. Inspirációs háttér	12
2.1. Közlekedési szabályok	12
2.2. Közlekedési táblák	13
2.2.1. Egyenrangú kereszteződés	13
2.2.2. Elsőbbségadás kötelező	13
2.2.3. STOP	14
2.2.4. Főút	14
2.2.5. Vasúti átjáró	14
3. Rendszerterv	15
3.1. A rendszer célja	15
3.2. Fő funkciók	15
3.3. Telepítés	16
3.4. Fejlesztéshez használt eszközök	16
3.5. Követelmények	17
3.5.1. Funkcionális	17
3.5.2. Nem funkcionális	17
4. Játékmenet	18
4.1. Szabad játék	18
4.2. Teszt játék	19
5. Saját szoftver	22
5.1. Fejlesztési idővonal	22
5.1.1. Tanulási időszak	22

5.1.2.	Szoftver alapok	22
5.1.3.	Unity alapok	23
5.1.4.	Legfrissebb verzió	23
5.2.	Menü	23
5.3.	Játékprogram	24
5.3.1.	Utak	24
5.3.2.	Járművek	25
5.3.3.	Kereszteződések	26
6.	Fejlesztési lehetőségek	27
6.1.	Textúrák és modellek	27
6.2.	Új pályák és szituációk	28
6.3.	Új szabályrendszerek	28
6.4.	Szabálylista	30
6.5.	Testreszabható feladatok	30
7.	Tesztelés	31
7.1.	Manuális	31
7.1.1.	Tesztesetek	32
7.2.	Egységteszt	32
	Összegzés	33
	Irodalomjegyzék	34
	Ábrajegyzék	35

Bevezetés

Sosem állt tőlem messze a számítástechnika. Már kiskoromban találkoztam személyi számítógéppel, mobiltelefonnal és egyéb programozható felületekkel. Gyerekkoromban nagy érdeklődést kezdtem el mutatni a robotika és a programozás iránt. Először még csak szakköri szinten csináltam egyszerű játékokat gyerekeknek kitalált felületen. Később elkezdtem komolyabban is érdeklődni a videojátékok körében és találtam is párat ami tökéletesem megfelelt délutáni tevékenységnek. Minden játéknál amit valaha játszottam felmerül bennem a kérdés, hogy vajon hogyan működnek a háttérben a dolgok. Mi az ami lehetővé teszi, hogy a játékot lássam és játsszam. Az egyetem előtt is tanultam már informatikát, de úgy érzem hallgatóként mélyült el leginkább a tudásom. Az órákon értettem meg igazán mekkora különbség van a játék és a játékfejlesztés között.

Mindenféleképpen olyan szakdolgozatot akartam amihez motiváció is köt. Akkoriban a számítógépes játékok közül is a versenyzős, autós téma fogott meg a legjobban, és nagyon elkezdett érdekelni az autók és a közlekedés világa. De ahhoz, hogy elmerülhessek benne kellett egy saját gépjármű, amihez pedig vezetői engedély szükséges. Így hát amikor szakdolgozati témát kellett választanom, akkor éppen a szabadidőm nagy részét a jogosítványom megszerzésére fordítottam. A szabályok megértése és megtanulása egyszerű volt, de az úton, a közlekedési helyzetekben való alkalmazásuk már sokkal bonyolultabb. Az úgynevezett „3 pontos” feladatok, amik egy KRESZ vizsga jelentős részét alkotják, erre alapulnak. Az ilyen feladványok játékos gyakorlására akartam megoldást találni, Mivel úgy éreztem, hogy a mostani tananyagok a témában nagyon nyersek és monotonok, ráadásul a legtöbb esetben egy egyszerű képet kell értelmezni rövid idő alatt ami még inkább megnehezíti a tanulók dolgát.

Egy olyan felületet szerettem volna létrehozni ami magába tudja foglalni egy közlekedési szituáció minden olyan elemét ami a tananyag részét képezi. Szerettem volna, hogy a program hasonlítson az előbb említett vizsgafeladatokra, hogy a megjelenés már ne egy újdonság legyen azok számára akik ilyen eszközzel szeretnének tanulni. Fontos hogy a részletek könnyen felismerhetőek legyenek, és az egész jelenet interaktív legyen, segítve az adott helyzetben érvényesülő szabályok megértését.

A Unity a C# programozási nyelvet használja, amit az egyetem során több ízben is tanultam és megismerhettem. A C alapú nyelvek többször is felbukkantak az egyetemi órákon így biztossá vált, hogy közülük fogok választani egyet. Végül tervem megva-

lósítására a Unity keretrendszert találtam a legalkalmasabbnak. Szakdolgozatomban bemutatom a fejlesztéshez használt technológiát, valamint a dolgozat inspirációs hátterét, a lépéseket amikkel a projekt haladt, és felvázolja a jövőbeli fejlesztéseket.

A szakdolgozat kódja, a hozzá tartozó build, és a játék működését bemutató videó a itt található:

Forráskód: <https://github.com/FourthRuffle/KRESZTraffic-thesis>

Build: <https://github.com/FourthRuffle/KRESZTraffic-thesis/releases/tag/thesis>

Videó: <https://youtu.be/Q9aeaCG2Be4>

1. fejezet

Technológiai áttekintés

A játékfejlesztés sok különböző technológiai fejlesztéssel büszkélkedhet. A modern játékok alapja a játékmotor, ez egy olyan szoftver ami barkácsasztalként funkcionál a fejlesztők kezeiben. A videojátékok rendszerei bonyolultak és rengeteg elemből állnak össze, azért használnak a programozók játékmotorokat mert az fejlesztői környezetek, azok a keretrendszerek amiket az évek alatt létrehoztak, már tartalmazzák a legtöbb olyan funkciót amik elengedhetetlenek lennének egy modern videojáték fejlesztéséhez. Ilyenek az animációk, a szimulált fizika, valamint a játékok háttérben futó logikája. A napjainkban legismertebb játékmotorok az Unreal Engine, Godot és a Unity.

A 20. század alkonyán a játékfejlesztők még sokkal kevesebben és kisebb erőforrásból dolgoztak, így sokkal gyakoribb volt, hogy a játékaikhoz saját motort fejlesztettek, mivel akkoriban nem sok kész, piaci játékmotor volt elérhető. Nagy segítség lehetett viszont, ha valaki meg tudott vásárolni egy kész kódbázist.

Ilyen volt például az *id Software* cég fejlesztette „id Tech” motor, melynek később több verziója is készült. Ezeket a motorokat használták az első meghatározó FPS (first person shooter) játékok mint például *DOOM*, *Quake*, *Wolfenstein* sorozatok alapjául.[1]

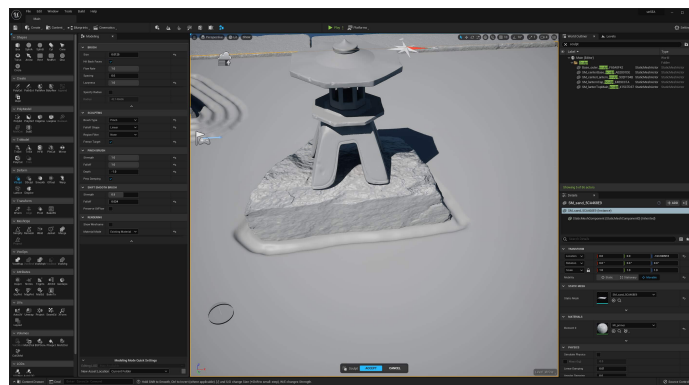


1.1. ábra. Korai képernyőkép a DOOM játékból (DOOM 1993) [19]

1.1. Unreal Engine

Az *Unreal Engine* az mai napig egyik legmeghatározóbb játékmotor. Az Epic Games játékfejlesztő cég 1998-ban készítette az *Unreal* nevű játékukhoz. Eredetileg az is az FPS genre kiszolgálására készült. Később viszont a TPP (third person perspective) féle kalandjátékok és rengeteg más 3D-s játék alapjául is szolgált, legyen az lövöldözős, kaland, nyílt világ, vagy történet műfajú. Manapság már nem csak játékfejlesztésre hanem más grafikai alkalmazások és fizikai szimulációk szemléltetésére is használják.

Az Unreal Engine olyan eszközöket és funkciókat tartalmaz, amelyek egyesítik az interaktív tervezést és a hagyományos technikákat, ez lehetővé teszi, hogy magával ragadó történeteket és játékelményt hozzanak létre vele. A fejlesztők meglepően élethű és lenyűgöző dolgokat az Unreal Engine dinamikus világításával, valós idejű render rendszerével és karakterkészítő eszközeivel. A motor méretezhetősége és alkalmazhatósága miatt főleg, nagyszabású, nyílt világú játékok készítésére használják leginkább funkciók révén, mint a *World Partition* vagy a *Blueprint* vizuális fejlesztés.[2]



1.2. ábra. Unreal Engine kezelőfelület (unrealengine.com) [20]

Az Unreal Engine rendszere teret ad a „real-time” interaktív alkalmazások és játékok fejlesztéséhez, egybefoglalja megjelenítést, a fizikai szimulációt, az animációkat és logikát. Ezek az integrált rendszerek összesége és kapcsolata teszi lehetővé a bonyolult virtuális világok létrehozását és futtatását. A motor a mai napig C++ nyelven működik.

Az alapvetően ingyenes szoftver legújabb, 2022-es verziójának forráskódja a GitHub-on is elérhető. Tanulóknak és iskoláknak teljesen ingyenesen letölthető és korlátlanul használható, viszont a kereskedelmi felhasználás jogdíj köteles, amit az Epic Games a bevétel 5%-aként határozott meg amennyiben az eléri az egymillió amerikai dollárt. [3]

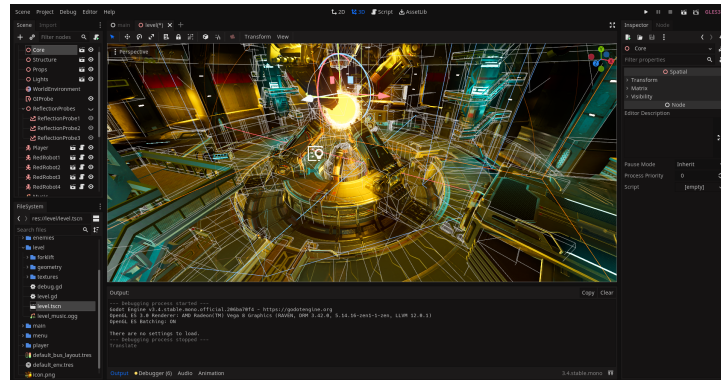
1.2. Godot

A Godot egy nyílt forráskódú játékmotor, ez lehetőséget ad minden fejlesztőnek, hogy szabadon használja és módosítsa vagy továbbfejlessze a már felépített rendszert. A

nyitott forráskódú szoftverek jellemzője, hogy a rendszer gyorsan fejlődnek az aktív és folyamatosan növekvő közösség miatt.

Fontos tulajdonsága, hogy a játékfejlesztés legmeghatározóbb elemeit, a jeleneteket, objektumokat, animációkat, és a játék logikáját egy saját integrált szerkesztői felület fogja össze. A fejlesztők minden lényeges részt, alkalmazáson belül tudnak kezelni, így lecsökken a külső alkalmazásokra való hagyatkozás igénye. A jelenetek és a csomópontok adják a motor magját. A csomópontokból épülnek fel a jelenetek miután a csomópontok egy szabályszerű csoportba rendeződnek.

A Godot motorjában 2D-s és 3D-s játékokat is alkothatunk, ehhez megvan a program fizikai és grafikai képessége is, valamint van benne lehetőség a érintkezések kezelésére is így komplexebb játékmeneteket is létrehozhatunk akár. Napjainkban a fejlesztés a 4.0-ás verziónál tart, ahol a 3D-s felületet igyekeznek felzárkóztatni a versenytársakhoz.



1.3. ábra. Godot kezelőfelület (Godot 4.6 2026) [21]

A motor nem csak egy programozási nyelvhez kötött, de a három leggyakoribb a C++ és C#, valamint a GDScript. A GDScript a Godot saját rendszeréhez optimalizált objektum orientált programozási nyelv, a Pythonhoz hasonló struktúrával. A vizuális programozás opciója itt is megjelenik, régebben a *VisualScript*-ként, ez viszont a 4.0-ás verzióban már nem elérhető. [4]

1.3. Unity

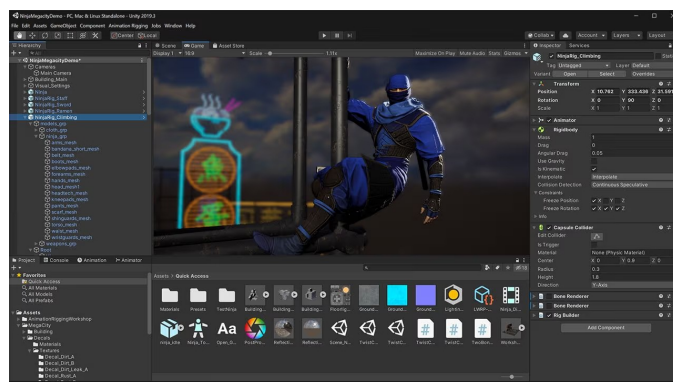
A Unity 2005-ben indult, és mára már az egyik legismertebb és legmeghatározóbb játékfejlesztési környezet. A letöltése és a használata ingyenes, viszont itt is megjelenik a nagykereskedelem, ahol már természetesen díjat kell fizetni a keretrendszer használatáért. A fejlesztőcég először *Over The Edge Entertainment* jelent meg, csak 2007-ben nevezték át *Unity Technologies*-ra. [5] A motor támogatása kiterjed a számítógépeken kívül a konzolokra és okostelefonokra is. A 2D-s és 3D-s alkalmazások készítésére is alkalmas fejlesztői környezet hamar elterjedt a mobilfejlesztők és a feltörekvő, egy vagy több személyes kis fejlesztőcsapatok körében. A Unity felépítése könnyen megérthető

és gyorsan tanulható, emellett rengeteg oktatóvideó és kurzus is segíti a kezdőket az alapok elsajátításában.[5]

A nagykereskedelmi díjazás általában nem probléma, a nagy fejlesztőcégeknek van annyi bevételük, hogy sok pénzt kifizessenek a licenszért. 2023-ban viszont nagy dráma volt a Unity áraival kapcsolatban amikor is bevezették a *Unity Runtime Fee*-t. A lényege az volt, hogy a fejlesztőknek a felhasználói telepítések száma után kellett volna részesedést fizetnie a *Unity technologies*-nak. Ez sokakat felbosszantott, és az internetes fórumok megteltek a felháborodott fejlesztőkkel. Később 2024-ben a felhasználói nyomás hatására bejelentették, hogy eltörlik a ezt a típusú díjazást és visszatérnek az ülésalapú díjazáshoz, ami azt jelenti, hogy fix havidíjat vagy éves díjat számolnak fel, minden olyan fejlesztő után aki hozzáfér a szoftverhez és annak szolgáltatásaihoz. A magánszemélyeknek továbbra is ingyenes maradt.[5]

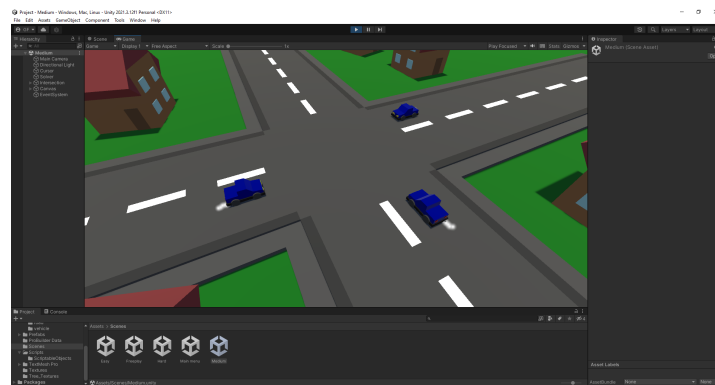
A Unity a C# programozási nyelvet használja alapként. Az objektumokhoz szkripteket csatolhatsz. A szkriptekben írod le a játék logikáját, ezután a szkripteket, hozzacsatolod a *GameObject*-ekhez, így azok megkapják a szkriptekben leírtakat mint tulajdonság. A szkriptek hasonlóan működnek mint a komponensek, funkciót adnak az adott objektumnak, legyen az jármű, a játékos karaktere, vagy akár egy egyszerű fa ami ki tud dőlni. A Unity objektumorientáltsága nagyon népszerűvé tette a játékfejlesztők köreiben. Sok híres játékfejlesztő cég döntött már emellett a keretrendszer mellett, de rengeteg apró projekt alapja is a Unity.

A Unity vizuális kezelőfelülete, az Editor, egy integrált fejlesztői környezet amit a játék elemeinek a kezelésére használhatunk. Itt majdnem mindenhez hozzáférünk ami játékunkhoz kell. Létrehozhatunk és szerkeszthetünk komponenseket, *GameObject*-ek segítségével felépíthetjük a játék világát. Az Editor több részből áll először is, az *Inspector* segítségével nyomonkövethetjük az összes elem tulajdonságait, manuálisan változtathatjuk az objektumokhoz csatolt komponensek értékeit.



1.4. ábra. Unity kezelőfelület (2026) [22]

Alul a *Project* és *Console* fül. A Project alatt látjuk a mappahierarchiát és bennük az összes rendelkezésre álló eszközt. Ezek az eszközök angolul *Assets* az összes olyan fájl vagy elem ami a programhoz tartozik, a jelenetektől a szkripteken át és a legkisebb textúráig. Itt kezelhetjük a *PreFab*-jeinket is, amik a játékhoz használt modellek. Ezek alkotják a 3D-s játékok karaktereit és környezetét. A másik fül a *Console*, itt tudjuk ellenőrizni a Unity üzeneteit mint például az esetleges hibák értesítőit vagy az állatlunk a szkriptekben leírt „logolás” eredményeit. A programban én is használtam egy *Feedback* rendszert ami minden számomra hasznos adat pillanatnyi állapotát írta ki nekem a konzolra.



1.5. ábra. Unity kezelőfelület (Saját projekt)

1.3.1. Komponensek

A Komponensek a Unity egyik fő résztvevői. A komponens tulajdonságai teszik lehetővé, hogy egy *GameObject* személyre szabható legyen. A *GameObject*-ek építik fel a játékokat, ezek a fő elemei a kész programnak. Sok komponens mint például a *Transform* vagy a *Collider*-ek, kevesebb funkciót ágyaznak magukba vagy rengetegszer felbukkanhatnak a program során. [6]. Unity a saját komponensek létrehozását is lehetővé teszi, ezáltal kivételes szabadságot adva a fejlesztőnek. Amennyiben ezeknek a komponenseknek őse a *MonoBehaviour* osztály, az összes többi *GameObject*-re rácsatolhatóvá válnak és így új funkciókat adhatunk az adott objektumnak. [7]

A *GameObject* a Unity Editor legfontosabb elemei. Minden tárgy, minden elem amit látsz a szerkesztői felületen *GameObject*-nek számít, a játék karaktereitől, a kamerák, fény és hangforrások, de az összes fa vagy házikó is mind közéjük tartozik. Magukban viszont még nem képesek semmire, „property” azaz tulajdonságokat kell adni nekik. A tulajdonságok mellé társulnak a komponensek amik funkcionalitást adnak nekik. Ilyen például a *Transform* komponens, amivel minden *GameObject* rendelkezik, ezzel határozzuk meg a méreteit, az elforgatását és a pozícióját.

Egy egyszerű kockának van még *Mesh*, és lehet például *Collider* komponense. A *Mesh Filter* a kocka 3D grafikai megjelenítésének az alapja. Lehet három vagy négyszög

alapú. Ide tartozik még a *Mesh Renderer* is ami filter geometriai adatait felhasználva rendereli a tárgy pozícióját a Transform komponens alapján. A Collider már sokkal inkább opcionális. Egy kocka esetében *Box Collider*t adunk neki. Ez a komponens a nevéből adódóan „collide” azaz ütközni tud más colliderekkel. Így működik a „hitbox”, egy autós játékban, mindkét autó rendelkezik colliderel, nem haladhatnak át egymáson és ha egymásnak ütköznek a az autók összetörnek.

Komponens csoportok is léteznek, ilyen például a *NavMesh*. A *NavMesh* egy mesterséges intelligencia alapú rendszer. Fő feladata a „Pathfinding” azaz útkeresés. Három fő része van, a felület, az ügynök és az akadály. A NavMesh feladata, hogy bejárja a felületet kikerülve az akadályokat és ezzel megtalálni a legmegfelelőbb útvonalat. A technika lényege, hogy a nyílt világban navigálás megoldása. [8]

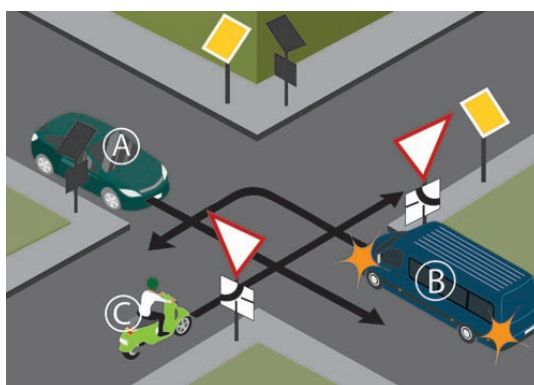
Egy másik nagy csoport a *Cinemachine* ami a projekt animációiért felel. Rengeteg különféle részből és komponensből áll, és a segítségével sokféle animáció hozható létre a környezeten belül. A projektben használt komponens a *DollyCart*[9] volt ami járművek megtett útjának a kezdőpontból a végpontba történő mozgás animációját biztosította. A járművekhöz csatolt *DollyCart* komponens képes az utakra rögzített animációs síneken végig haladni, így azt a látványt keltve, hogy a jármű áthaladt a kereszteződésen.

2. fejezet

Inspirációs háttér

2.1. Közlekedési szabályok

A Közúti Rendelkezők Egységes Szabályozása (rövidítve: KRESZ) az a közlekedési szabálygyűjtemény amely a Magyarország területén levő közutakon és közforgalom előtt el nem zárt magánutakon folyó közlekedést szabályozza. A közúti forgalom rendjét már a 19. században elkezdték meghatározni. Az első magyar KRESZ egy 1910-ben született belügyminiszteri rendelet ami az azt egy évvel megelőző *Párizsi egyezmény* alapjaira készült. Ekkor még csak töredéke volt a mai szabályrendszernek. Ami viszont már ebben a korban és megjelent a KRESZ-ben az az autók műszaki vizsgálata és rendszámozása, a gépjárművezetők képzése, valamint az első négy közúti jelzőtábla. 1941-ben pedig bevezették a jobboldali közúti közlekedést és ezzel együtt az úgynevezett „Jobbkéz szabályt”, inentől kezdve vált igazán fontossá a utak alárendeltsége, az elsőbbségadás és az áthaladási sorrend. Ezek nélkül a közlekedés valóságos káosz lenne.



2.1. ábra. Három pontos feladat
(szuperjogsi.hu) [13]

2.2. Közlekedési táblák

Manapság már több mint 100 közlekedési tábla van az utakon. Minden tábla különböző jelentéssel bír és sok csoportba lehet őket sorolni. A közlekedési táblák, segítenek, figyelmeztetnek, útba igazítanak. Az internet és a GPS rendszerek felvirágzása előtt már mindössze a közúti táblákból tájékozódva el lehetett jutni szinte bárhova a világban, és a legfontosabb, hogy biztonságban.



2.2. ábra. KRESZ táblák [14]

2.2.1. Egyenrangú kereszteződés

Ez a tábla a figyelmeztető táblák közé tartozik, az ilyen táblák többnyire fehér alapú piros szélű háromszögek a az élükre állítva. Ebben az esetben a háromszög közepén egy fekete „X” van ezzel utalva a kereszteződésre. Az ilyen kereszteződésekben mindkét út ugyan azzal az alárendeltségi szinttel bír, így a rajtuk haladókat nem korlátozza tábla. Ez esetben sem mehetnek viszont egymásnak, erre találták ki a jobboldali közlekedéssel társuló jobbkéz szabályt, ami kimondja, hogy ilyen feltételek mellett a járműveknek a nekik jobbról érkező másik járműnek elsőbbséget kell adniuk. Ez a szabály a baloldali közlekedéssel rendelkező országokban fordítva működik.

2.2.2. Elsőbbségadás kötelező

Az *Elsőbbségadás kötelező* tábla sokszor társul a *Főút* táblával. Ez azért fordul elő, mert ez a tábla arra figyelmeztet, hogy egy felsőbbrendű útszakasz következik, és a tábla melletti járműnek elsőbbséget kell adnia a főúton haladóaknak. Itt még nem kötelezik a járműveket megállásra, így abban az esetben, ha nincs senki akinek elsőbbséget kell adnia, egyszerű lassítással és körültekintéssel léphet be a kereszteződésbe.

2.2.3. STOP

A *STOP! Elsőbbségadás kötelező* az amire a legtöbben emlékeznek és asszociálnak ha közlekedési tábláról van szó. Piros nyolcszögletű tábla ami hasonló jelentéssel bír mint az előző, viszont ebben az esetben már kötelező a megállás is. A legtöbb STOP tábla útburkolati jellel is el van látva, ennél a jelnél minden esetben meg kell először állítani a járművet amikor egy ilyen táblához érkezünk. Amennyiben a kereszteződés nem belátható, a vonalnál megállást követően előrehaladhat a másik úttest széléig, hogy onnan hajtsa végre a becsatlakozást.

2.2.4. Főút

A *Főút* a felsőbbrendű utat jelzi egy sarkára állított fehér alapon sárga-fekete táblával, általában kétirányú és egy-egy sávból áll. Az ezen haladó számíthat arra, hogy a mellékutakról becsatlakozni kívánó más járművek megadják neki az elsőbbséget. Ettől függetlenül ajánlott odafigyelni a mellékutakra, vagy esetleg más főutak kereszteződéseire, habár ezek már többnyire közlekedési lámpával is el vannak látva.

2.2.5. Vasúti átjáró

A vasúti átjáró a legveszélyesebb hely a közlekedéskultúrában. Tábla, sorompó és fény-sorompóval biztosítják, hogy ne legyen jármű a vasúti síneken amikor a vonat áthalad. Ez a tábla nem egy szokványos kereszteződésre figyelmeztet. Ezeken a helyeken van egy minimum sebesség, legalább 5 km/h-val kell áthaladnia a járműveknek az átjárón. [12]

3. fejezet

Rendszerterv

3.1. A rendszer célja

A szakdolgozat célja egy olyan program létrehozása, amiben a KRESZ vizsgák és tananyagokban szereplő „3-pontos” feladatokat játékos módon érthetőbbé, átláthatóbbá és tanulhatóbbá teszi. Alapjaiban életszerű közlekedési szituációk szimulálása, és teszt szerű megoldása. Fontos, hogy a játékos tudja tesztelni a tudását, a játék pedig interaktív és rezponzív legyen.

3.2. Fő funkciók

A játékos két mód közül választhat, az elsőben véletlenszerűen felépített szituációkat kap amiket meg kell oldania. A megoldás folyamata, hogy először megvizsgálja az összes feladatban szereplő járművet, és a kereszteződést. A kereszteződés táblákkal lehet ellátva amik jelzik az adott út esetleges alárendeltségét. Az utak után megvizsgálja a járműveket, a program megmutatja, hogy az adott járművek melyik irányba szeretnének továbbhaladni és melyik útra fognak érkezni.

Ezekből az információkból a játékosnak ki kell találnia, hogy mi a feladat helyes megfejtése. A program is ugyan ezek információk birtokában egy algoritmus segítségével megoldja a feladatot. A feladatmegoldáshoz a játékosnak rá kell kattintania a kiválasztott járműre, ami helyes lépés esetén elindul és elhagyja a kereszteződést. Ezután a játékos ezt a folyamatot ismételteti ameddig el nem fogynak a járművek. Minden helyesen megoldott feladat után jár egy pont. A játékos annyi feladatot oldhat meg amennyit akar, így gyűjtheti a pontokat.

A második módban szintén ilyen feladatokat kell megoldani viszont egy véges ellenőrzött környezetben. Összesen 30 feladatból 90%-os eredményt kell elérni a teszt teljesítéséhez. A teszt 3 különböző nehézségi fokozatból álló szituációkat használ, arányosan elosztva. Minden tesztet 10 darab könnyű, 10 darab közepes és 10 darab nehéz

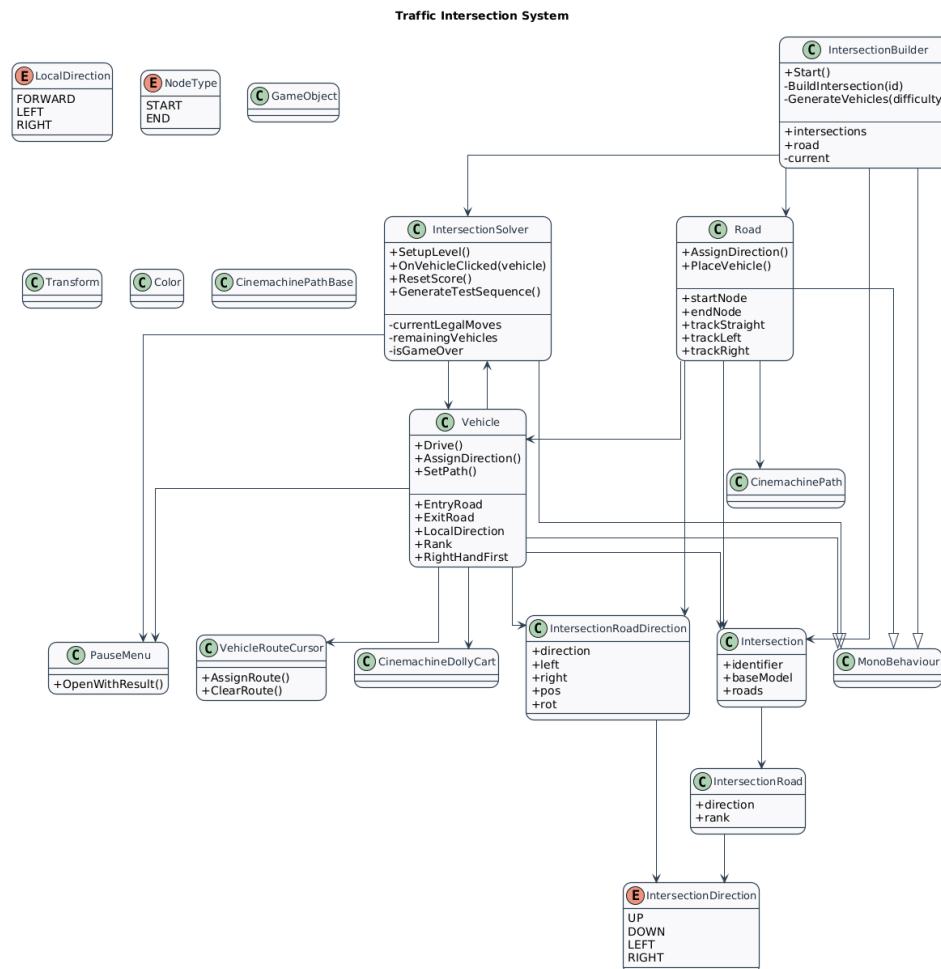
szintű feladvány alkot.

3.3. Telepítés

A nem szükséges telepíteni a játékszoftvert, egyből a kicsomagolás után indítható a mappában található alkalmazással.

3.4. Fejlesztéshez használt eszközök

- Unity: A játék készítéséhez használt fejlesztői környezet. Verzió: 2021.3.12f1
- Visual Studio: A programozói környezet, amely Unity támogatással rendelkezik. Verzió: Community 2022
- Visual Studio, WinForms: Vizuális fejlesztőkörnyezet a Visual Studio-n belül. A program kezdeti algoritmusainak a C# nyelvben való megvalósításra szolgált.



3.1. ábra. PlantUML ábra a rendszer működéséről [23]

3.5. Követelmények

3.5.1. Funkcionális

- KRESZ alapú szabályrendszer: Egy olyan háttér algoritmus ami a szituációban szereplő objektumokat kezelni. Az algoritmus áthaladási sorrendet állít fel ami egyben a feladvány helyes megoldását is meghatározza.
- Interaktív játékos tér: A feladatot alkotó szereplők kattinthatóak legyenek és rendelkezzenek animált mozgással. A játék értesítse a felhasználót a hibás lépésről, vagy helyes megoldásról.
- Szabad játék: Végtelen ismétlődő randomizált feladványok ahol a játékos gyakorolhatja a közlekedési szituációkat.
- Nehézségi szintek: Különböző nehézségű feladványok a könnyebben követhető tanulási és gyakorlási görbéhez valamint a monotonitás elkerüléséhez.
- Teszt felület: Egy véges „Teszt” mód ahol a gyakorolt feladatokat adott megadott összesített sikerességi rátával kell megoldani a teszt teljesítéséhez. Támpont a tudás és gyakorlat fejlődésében.
- Pontrendszer: Mind a szabad játéknak, mind a teszt felületnek megfelelő pontrendszer ami a játékos eredményét rögzíti folyamatosan.

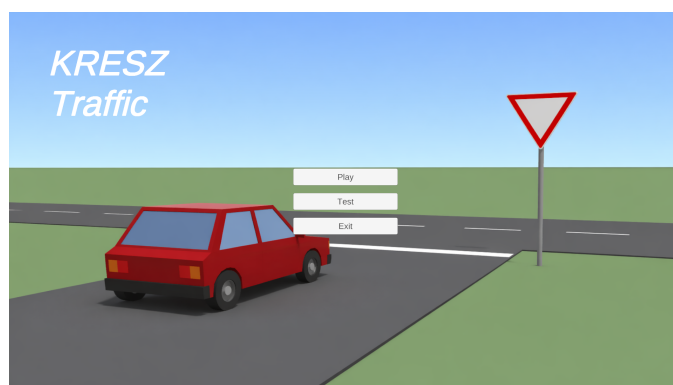
3.5.2. Nem funkcionális

- Egyszerű letisztult kinézetek: Az átláthatóság érdekében kell egy szimpla dizájn ami egyértelműsíti a részvevőket.
- Pálya változatosság: A kereszteződések felépítésébe lehet variációt építeni de ez a működésben nem befolyásolja a programot.

4. fejezet

Játékmenet

Az indítás után az első megálló a főmenü. Itt választhatjuk ki, hogy a „Play” gombra nyomva egy szabad játékba kezdünk, vagy a „Teszt”-el egy tesztet indítunk, ahol megmérgethetjük saját tudásunkat a gyakoroltak alapján.



4.1. ábra. A játék Főmenüje

4.1. Szabad játék

A szabad játék egy végtelen játékmód. Itt az összes elérhető variációból véletlenszerűen kiválasztott feladatokat kap a játékos amiket a közlekedési szituációban résztvevő autókra kattintva kell megoldani. A program logikai algoritmusai előre megoldja a feladatot, és minden kattintás után eldönti, hogy az adott lépés a szabályoknak megfelelő volt-e. Ha nem a feladatnak vége és dönthetünk, hogy folytatjuk a játékot vagy visszalépünk a menübe. Amennyiben az elsőnek megjelölt járműre kattint, az elhagyja a kereszteződést és a program újraszámolja a megoldást. Minden helyes lépés után a program új szituációként tekint a felállásra így mindig az az autó következik ami a közlekedési szabályok szerint a következő a sorban.

Minden sikeresen megoldott feladat után egy pont jár, az ide tartozó pontrendszer a játékos aktuális játéka alatt helyesen megoldott feladványait tartja számon egész



4.2. ábra. Hibásan megoldott feladat a szabad játékban



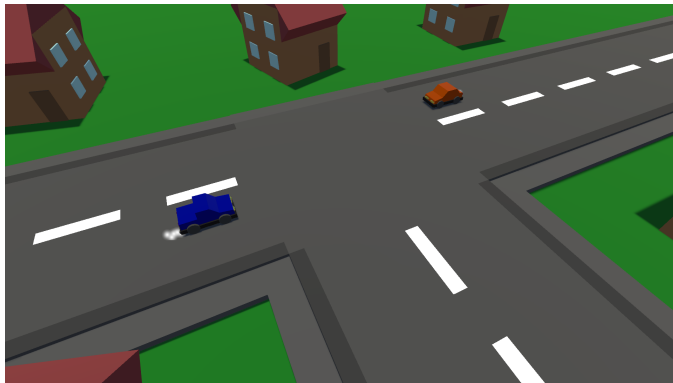
4.3. ábra. Sikeresen megoldott feladat a szabad játékban

addig még a játékot be nem fejezik. A szabad játék lényege a tanulás, végtelen idő és próbálkozás áll a játékos rendelkezésére, így nincs nyomás mint egy dolgozatnál vagy vizsgánál. A játékosnak van ideje kielemezni az összes szituációt és lépésekben rájönni a megoldásokra. Segít megérteni és alkalmazni a közlekedési szabályokat. Ha felhasználó már magabiztosan tud feladatot megoldani a szabad játék módban akkor érdemes továbblépni a következő játékmódra, hogy tesztelje a megszerzett tudását és gyakorlatát.

4.2. Teszt játék

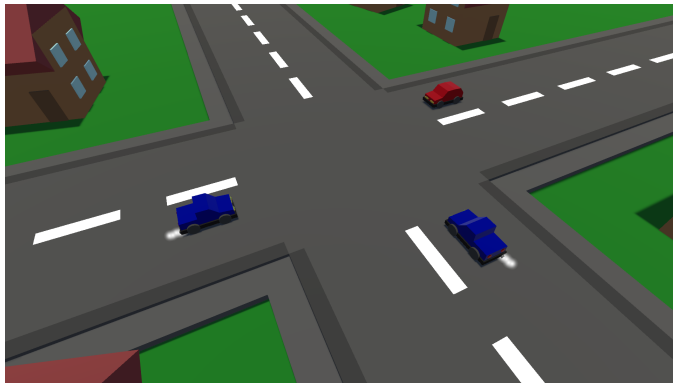
A „Teszt” gombra kattintva egy teszt játék veszi kezdetét. A játék alapja ugyan az viszont itt már a *KRESZ vizsga* [12] mintájára, adott darab feladványt kell megoldania a játékosnak. Egy teszt feladatai már meghatározottabbak mint a szabad játék szituációi. A Teszt indításakor a program arányosan feltölti a feladatok listáját a három nehézségi szinthez (könnyű, közepes, nehéz) tartozó szituációkkal.

– Könnyű:



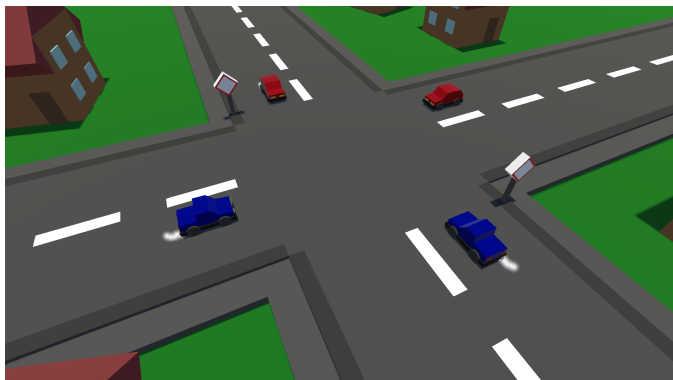
4.4. ábra. Könnyű szintű feladat

– Közepes:



4.5. ábra. Közepes szintű feladat

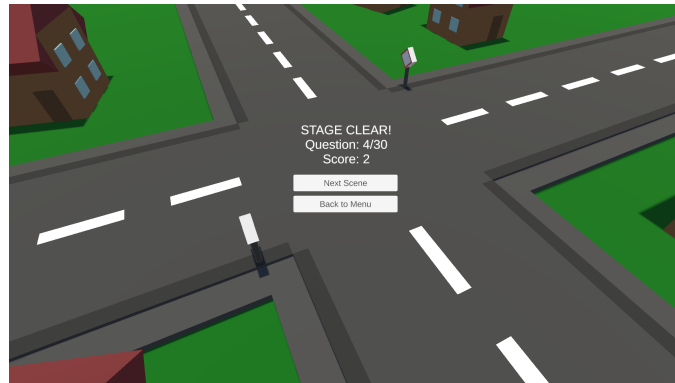
– Nehéz:



4.6. ábra. Nehéz szintű feladat

A nehézségi szintek garantálják, hogy a teszt során az összes típusú kereszteződéssel és kevesebb vagy több érkező autóval rendelkező szituációval is találkozik a játékos. A

tesztek hosszúsága és elosztása egyenlőre a forráskódban változtatható. A szabad játékhoz hasonlóan minden helyesen megoldott szint után 1 pont jár, itt viszont már más funkciója is van a pontoknak. A teszt sikeres teljesítéséhez 90% fölött kell teljesíteni, tehát 10 feladatból 9-et kell jól megoldani és minden autót kijuttatni a kereszteződésből. A tesztek legfőbb funkciója a gyakorlás.



4.7. ábra. A Teszt játék köztes menüje egy sikeres feladatmegoldás után

5. fejezet

Saját szoftver

5.1. Fejlesztési idővonal

A programom teljes kifejlesztése összesen 2 évet vett igénybe. Ez idő alatt voltak aktívabb és passzívabb időszakok is, volt amikor egy-egy áttörés után rövid időre megnövekedett az elért mérföldkövek száma. A program több alakot is öltött.

5.1.1. Tanulási időszak

Az első félév alatt a program elég csekély formában volt jelen. Ebben az időszakban a fókusz a projekthez használt programozási nyelv megértésén és elsajátításán volt. Dokumentációkat és rendszerterv példákat írtam. próbáltam először magam előtt papírra vetni a gondolataimat, hogy aztán azokat összegezve egy kész fejlesztési tervet tudjak összerakni.

Elkészültek a program első tervei amik később támpontot adtak a fejlesztés során. Már itt előre sok elméleti problémára próbáltam megoldást keresni, ötletelni, hogy ha esetleg ezek a problémák később felmerülnének, ne érjen akkora meglepetés.

5.1.2. Szoftver alapok

A program első verziója még teljesen a *Visual Studio*-ban, egy „WinForms” felületre készült. Itt jött létre a program alapjául szolgáló algoritmus első, primitív verziója, ami a szituációk előre megadott adataiból volt képes kiszámolni az áthaladás megfelelő sorrendjét. Ez az algoritmus ugyan azokat a paramétereket tudta kezelni mint a későbbi változatok. Ebben az állapotban még csak az alapszabályoknak megfelelően működött, nem tudta még kezelni a nem teljesen lineáris megoldásokat. A kiválasztott paraméterek alapján felállított egy sorrendet és azt adta vissza a kezelőfelületen, nem volt még vizuális megjelenítés, csak számok és szavak. Ez a program lett később a feedback rendszer alapja mivel hasonló módon kiíratások alapján működött.

5.1.3. Unity alapok

A második félév végén nagy áttörés történt. A program átköltözött a Unity keretrendszerébe. Az eddig kifejlesztett algoritmus megkapta az interaktív felületet az első vizuális megjelenítését. Ez már a GitHubra is felkerült.

A harmadik félévben rájöttem, hogy habár a programíráshoz már megvan a tudásom, a Unity sajátos eszközkészletét még messze nem ismerem eléggé. Tanuló anyagokat kerestem a témában, gyakorló projekteket hoztam létre, és teljesítettem egy Unity specifikus kurzust is, hogy a megfelelő szintre hozzam a tudásomat, mindemellett folyamatosan integráltam az ezalatt tanult módszereket és újdonságokat a programba.

Először még csak placeholdereket használtam, lapos egyszínű utak és színes kockák járművek helyett. Itt még csak az volt a lényeg, hogy az algoritmus által generált megoldást vizualizálni tudja és a számok és szavak halmazát lecseréljem a egy kezdetleges szimulációra. Ennél a verziónál jelent meg először a „Játék” funkció, ahol már működött a kattintás alapú feladatmegoldás és a járművek és utak információinak a közlése a játékos felé.

5.1.4. Legfrissebb verzió

Az utolsó pár hónapban is sok módosításon esett át a játék. Elkészült egy kezdetleges feedback rendszer ami a későbbiekben lényegesen megkönnyítette a hibakeresést és a javítást, hiszen a konzolon folyamatosan jelentek a „real time” adatai az aktív feladatnak. Később pedig az eredetileg placeholdernek szánt modellek is lecserélésre kerültek, és megkapták az animációkat. [9]

Az algoritmus finomodott és immár képessé vált a nem lineáris sorrendű megoldások kezelésére. Ez gyakorlatban annyit jelent, hogy az algoritmus folyamatosan újrvizsgálja a szituációt minden lépés után, és megállapítja a következő helyes válaszlehetőségeket. Erről még bővebben később.

5.2. Menü

A játékban kétféle menü van. A Főmenü és a köztes játékmenü.

– Főmenü, egyszerű 3 gombbal rendelkezik:

- Play: Elintítja a szabad játék módot.
- Test: Elindítja a teszt játék módot.
- Exit: Kilép a játékból, bezárja azt.

– Játékmenü

- Next Scene: Tovább lép a következő feladatra.
- Back to Menu: Kilép a főmenübe. Ez mindkét játékmód esetében az alap helyzetbe helyezi az előrehaladást.

5.3. Játékprogram

A projekt fő része, játék alapjaként szolgáló program, 3 fő részből áll:

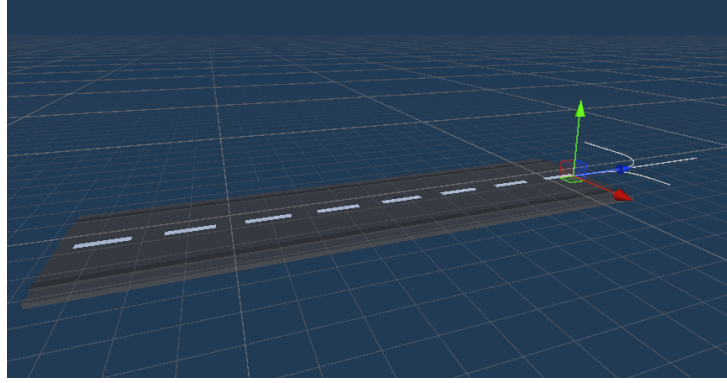
- Utak
- Járművek
- Kereszteződések

Az utak és a járművek alkotják a kereszteződéseket, utóbbiak pedig az előbbiekből elvégzik a szituációk létrehozásához kellő számításokat, és kezelik a megoldó algoritmust.

5.3.1. Utak

Elindítjuk a játékot, a jelenet futni kezd, a *Road* osztály az első megálló.

- Létrehozza a *Node*-okat.
 - *StartNode*: Az út objektumok jobb oldalán található Node. Ez a járművek kiindulási pontja, itt jelennek meg először.
 - *EndNode*: Az út bal oldalán található Node. Ide érkeznek a járművek, ez a kilépési pontjuk. Ez a Node határozza meg az adott jármű irányát.
- Megjelöli a felhasznált út objektumok irányát. Innen érkezik az az információ, hogy a jármű melyik utak Start- és EndNodejai közül választhat.
- Kiosztásra kerülnek az út rangok. Később ez alapján érvényesülnek az elsőbbség, és jobbkéz szabályok. Ha egy út magasabb rangú mint a szomszédja akkor az azon úton haladó jármű elsőbbséget élvez az alacsonyabb rangú úton állóval szemben. Ha az utak egyenrangúak, a jobbkéz szabály érvényesül.
- A *PlaceVehicle* metódus a *Vehicle* osztályból beérkező adatokkal egy járművet hoz létre, amit elhelyez az út *StartNode*-ján.

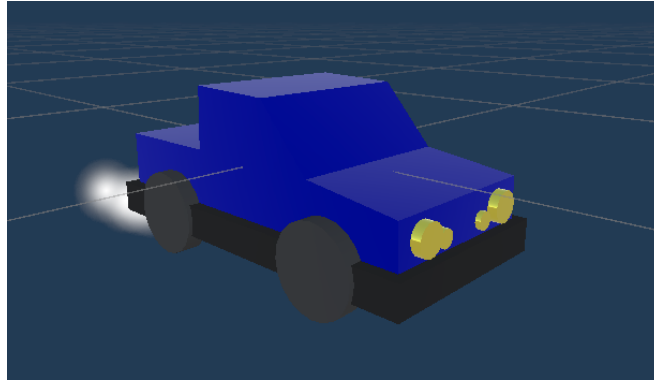


5.1. ábra. Egy út objektum felépítése

5.3.2. Járművek

A második megálló a *Vehicle* osztály.

- Létrejön egy jármű objektum, és a következő paramétereket kapja meg:
 - *Kiindulópont*: Egy olyan *StartNode* ami szabad.
 - *Irány*: Lehet előre, balra vagy jobbra.
 - *Cél*: Az alapján, hogy melyik irányba szeretne haladni, megkapja az annak megfelelő *EndNode* adatait. *EndNode* lehet közös, *StartNode* nem.
 - *Jobb szomszéd*: Megvizsgálja van e tőle jobbra másik jármű, hiszen egyenrangú kereszteződés esetén el kell engednie.
 - *Rang*: Ez a járművek rangja, a játék közben folyton változik ahogy a járművek megközelítik, majd elhagyják a kereszteződést. Ez adja meg az áthaladási sorrendet. Azért kell folyamatosan frissülnie, mert némelyik szituációban lehet több aktuálisan helyes lépés is, valamit egy lépés után lehet olyan jármű aki elveszíti a szomszédját.
 - Az animációkhoz használt komponensek is itt csatolódnak az objektumokhoz. [7]

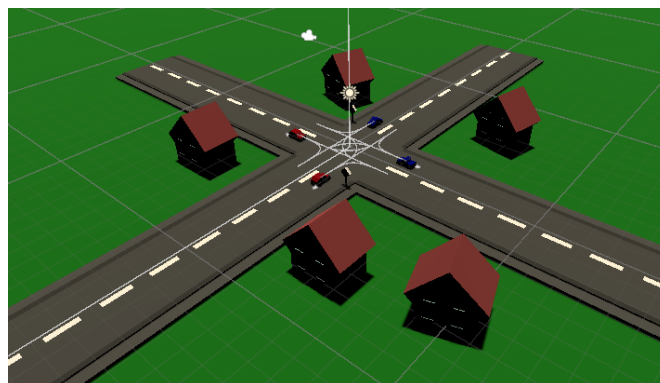


5.2. ábra. A járművek egyik modellje

5.3.3. Kereszteződések

A kereszteződéseknek két része van. Az *IntersectionBuilder* osztály és az *IntersectionSolver* osztály.

- *IntersectionBuilder*: „Megépíti” a kereszteződések. Felhasználva a kész út és jármű objektumokat és azok paramétereit, az elemeket összerakva vizualizálja a kész *Intersection* objektumot. Egy ilyen objektumnak legalább két járműből és három útszakaszból kell állnia, hogy megfelelő kereszteződést alkosson.
- *IntersectionSolver*:
 - Kezeli a megoldó algoritmust, a folyamatosan frissülő áthaladási sorrendet és az alapján Log-ol a konzolra is.
 - Kezeli a rövid és hosszútávú eredményeket a pontszámrendszerrel a szabad játék és a teszt játék esetében is.



5.3. ábra. Egy éppen aktív kereszteződés

6. fejezet

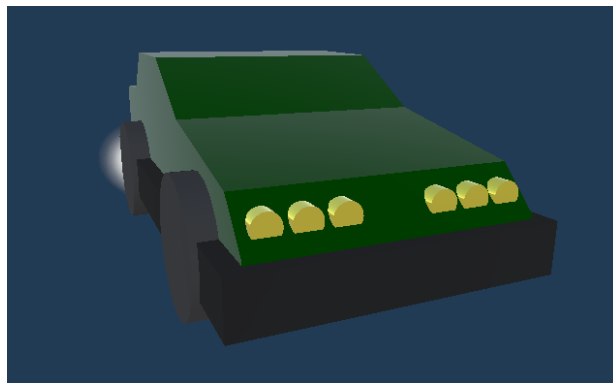
Fejlesztési lehetőségek

Rengeteg potenciál van még a programban. Sok olyan ötletem volt és van ami még nem valósult meg, de elméletben már létezik.

6.1. Textúrák és modellek

A projekt alatt sokkal több figyelmet fordítottam a program működési folyamatainak megbízhatóságára mint a megjelenésére, így van még pár olyan része a játéknak amire ráférne egy ráncfelvarrás:

- Autók: A játék kis szereplői, a közlekedésben résztvevő járművek habár aranyosak a maguk alacsony bonyolultságú kinézetükkel, sokat hozzátenne a játékélményhez az alaposabban kidolgozott autó modellek alkalmazása. A fejlesztendő textúrák és modellek közül ez számomra a legfontosabb, és ez egy hozzáértő új csapattag bevonásával könnyen megvalósítható lenne.



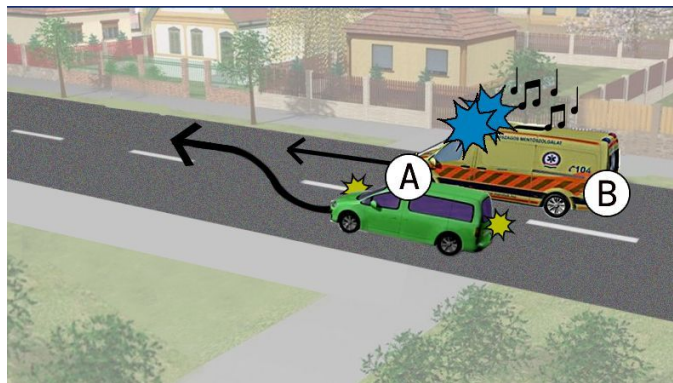
6.1. ábra. A járművek egyik modellje, amit lehetne fejleszteni

- Gombok és feliratok: A játék legtöbb felirata és gombjai egyszerű kinézettel rendelkeznek. Ezeknek a cseréje szintén nagyon feldobná a játékot és jobb esztétikai élményt adna.
- Környezet: A játékkörnyezet legalább annyira fontos mint a résztvevő autóké. A környezet kidolgozottsága egyeznie kell a járművekével, hogy az autók ne tűnjenek ki a helyzetből jobban mint kellene.

6.2. Új pályák és szituációk

Szeretném, hogy a játék ne legyen túlságosan egyoldalú. Ennek az eléréséhez a jövőben új pályákat tervezek hozzáadni amik egy fokkal feljebb emelik az alap kereszteződés alapú pályákat.

A kereszteződések mellett a következő lépés a kétirányú úttestek, körforgalmak és lámpás kereszteződések, esetleg további résztvevő járművek integrálása lesz.



6.2. ábra. KRESZ feladat mentőautóval [17]

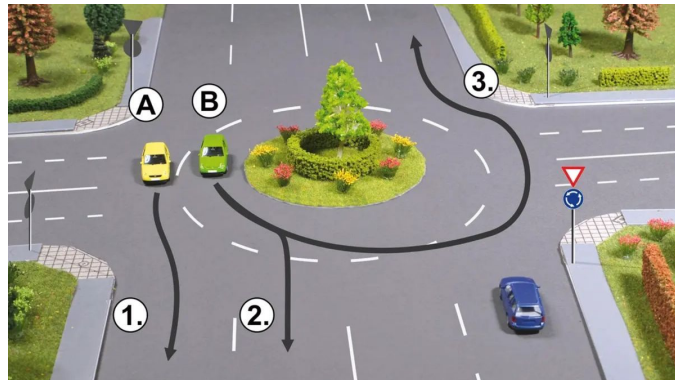
A nehézségi szintet tovább emelve, a járműveknek *Collidert* adva az áthaladási animációkkal egyszerű baleseteket is szimulálhatnánk, ami türelemre és odafigyelésre is tanítaná a játékost.

6.3. Új szabályrendszerek

Az új kétirányú, körforgalom és lámpás kereszteződések beépítésével új szabályrendszerekre is szükség lesz.

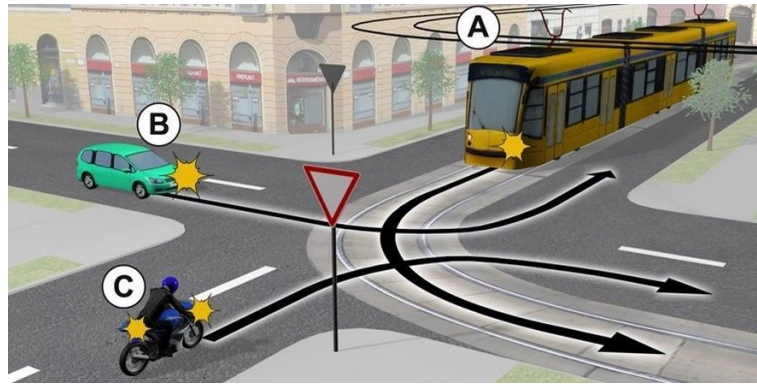
- Kétirányú úttest: Az ilyen típusú pályához kell egy olyan algoritmus ami tudja szimulálni az előzés, kikerülés, úttest elhagyása és a megkülönböztető jelzéssel haladó jármű érkezését, és az ezek által létrehozott szimulációban a kereszteződésekhez hasonlóan áthaladási sorrendet és megoldást generálni.

- Körforgalom: A Körforgalom egy egyszerű de annál furfangosabb és fontos része a közúti közlekedésnek. A beépítéséhez egy külön elsőbbség adási rendszer, és az irányjelzés és az azt figyelő algoritmus is része kell, hogy legyen.

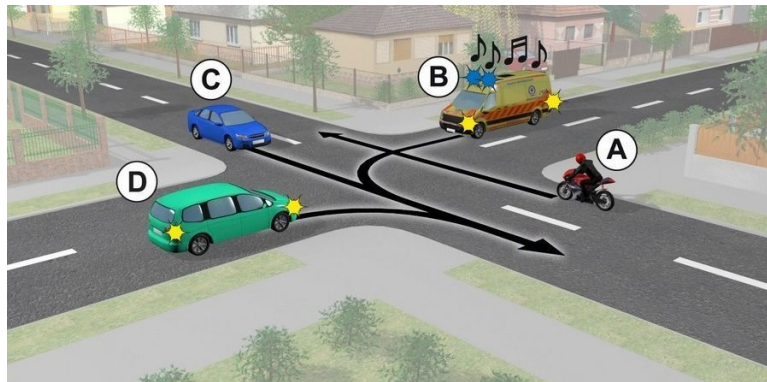


6.3. ábra. KRESZ feladat körforgalommal [18]

- Lámpás kereszteződés: A lámpás kereszteződés a mostanihoz hasonló de egy lépcsőfokkal ellenőrzöttebb rendszer, viszont fontosnak érzem, hogy a későbbiekben ez is a része legyen a programnak.
- Új járművek: A KRESZ-ben gyakran előfordulnak speciális járművek és résztvevők amikre a forgalom különös figyelemmel vagy akár konkrét szabályokkal reagál. Ilyen például:
 - Vonat: Vasúti átkelőhely.
 - Villamos: Kereszteződés villamossínnel, opcionális érkező villamossal.
 - Megkülönböztető jelzés: Rendőr, mentő, tűzoltó! A kereszteződésekbe és többi pályafarmába is opció.
 - Gyalogos: Sok esetben nem teljesen egyértelmű, hogy egy gyalogosnak mikor van elsőbbsége az autókkal szemben, és ez könnyen bezavarhat egy-egy szituációban, így extra nehezítésként a gyalogosok jelenlétét is lehetne adni a feladathoz.



6.4. ábra. KRESZ feladat villamossal [15]



6.5. ábra. KRESZ feladat mentőautóval [16]

6.4. Szabálylista

A játékon belül egy olyan felület, ahol a játék során fellelhető közlekedési szabályokat magyarázattal és illusztrációval vannak feltüntetve. Így ezeket a szabályokat a programon belül tanulmányozhatja a felhasználó, ha valamelyiknek szeretne utánanézni esetleg egy elrontott feladat után.

6.5. Testreszabható feladatok

Még az első bővítési ötleteim között volt a szituációk testreszabhatósága. Ez azt jelenti, hogy egy külön játékmódban a felhasználó beállíthatja, hogy:

- Milyen kereszteződés vagy közlekedési helyzet legyen a szituáció alapja.
- Milyen és hány résztvevője legyen az adott feladatnak.
- Az adott résztvevők milyen irányban haladjanak tovább, rendelkezzenek e bármilyen plusz joggal.

A beállítások véglegesítése után a játékos megoldhatja a saját szituációját aztán pedig új feladat létrehozásába kezdhet.

7. fejezet

Tesztelés

A programozás egyik elengedhetetlen része a tesztelés. A projektem fejlesztése során kétféle tesztelési módszert alkalmaztam: Manuális és egységtesztek, másik nevén *Unit tesztek*.

7.1. Manuális

A manuális tesztelés során olyan funkciókat vizsgálok, mint például a bemenet, megjelenítés, vagy a menü különböző gombjainak működése. A fejlesztés közben folyamatosan csináltam manuális teszteket. Minden új funkciót rögtön tesztelni kell, hogy megfelelően működik, így nagyon gyorsan kiszűrhetőek az esetleges hibák.

Később már végeztem nagyobb, teljes rendszer szintű manuális teszteket is, ahol egyben az egész játékot teszteltem, és nem csak a legfrissebb funkciókat. Ez azért fontos mert egy hosszabb fejlesztés után ha egy-egy funkció jól is működik, nem akadályozhatják egymást.



7.1. ábra. Manuális tesztelés (Teszt játék, nem elég pont)

7.1.1. Tesztesetek

Manuálisan teszteltem a kattinthatóságot és a kijelzőket. Több alkalommal is végigismételttem a feladatok megoldását, leellenőrizem, hogy a járművek minden kattintásra válaszoljanak, és a jelzők mindig megjelenjenek ha a kurzor a az adott járműn van.

Hasonló tesztsorozaton esett át az animációs rendszer. Itt leginkább a be- és kilépőpontok helyét, Az animációk lépéssebességét valamint, hogy megfelelő sínre csatlakozik e a jármű *DollyCart* komponense.

A pontrendszer manuális tesztelése volt az amivel a legtöbb időt töltöttem. A szabad játék pontrendszere egyszerű ezért a pontnövekedés sikeres megoldás esetén, és stagnálás hibás megoldás esetén alaphelyzetek mellett elég volt pár szélsőséges példát tesztelni. A teszt játék módnál több tesztelési lehetőség van. Vizsgálni kellett, hogy az eredmény elbírálása nagyobb feladatszámnál is pontos e, valamint, hogy a sikeres és sikertelen tesztek minden esetben megfelelnek e a valóságnak.

Tesztelni kellett a feedback rendszert. Ezt is több alkalommal, minden új adat implementálása után újra meg kellett ismételni, hogy minden aktív adat megfelelően megjelenik és érvényesül. A feedback rendszer nyújtotta az alapot a többi manuális teszt ellenőrzőpontjaként.

7.2. Egységteszt

Az egységtesztek olyan tesztek amiket a programunk adatokkal és változókkal foglalkozó részére írunk. Ezek a tesztek automatizáltak és azt tudjuk velük ellenőrizni, hogy a szoftver teljesíti e a feladatait.

A Unity keretrendszerben ezt egy külön package telepítésével tehetjük meg, a *Test Framework*-el. A kezeléséhez megnyitjuk a tesztelő felületet, azaz a *Test Runner*-t. Két opciónk van:

- Play Mode: Itt olyan tesztek tudunk csinálni amelyek futási időben dolgoznak. Ilyenkor a keretrendszer elindítja a játékot és úgy futtatja a tesztek.
- Edit Mode: Ebben a módban azok a kódrészek tesztelhetők amelyek nem foglalkoznak a futásidő alatt aktív objektumokkal.

Összegzés

A szakdolgozat végére úgy érzem, hogy az elképzelésem megvalósulni látszanak a kész programban. Eleinte úgy gondoltam, hogy sokkal egyszerűbb lesz, viszont ahogy haladtam előre a fejlesztésben, az újabb és újabb akadályok megmutatták mennyi belefektetett energiát is igényel egy ilyen program elkészítése. Rengeteg ötletem lenne még a program fejlesztésére, hogy milyen plusz funkciókat és elemeket tudnék beleépíteni a játékba. Ennek ellenére elégedett vagyok a program mostani állapotával is, hiszen mindig jó ha van hova fejlődni.

Tanulás szempontjából nagyon hasznosnak éreztem ezt a projektet. Rengeteg önfejlesztésen kellett átesnem, hogy elsajátítsam azt a tudást ami egy ilyen program fejlesztéséhez elengedhetetlen volt. Sok különböző programozási megoldást ismertem meg, és jobban megértettem az általam használt keretrendszerek működését is. Az 5. fejezetben leírtakból szeretnék a lehető legtöbbet valamilyen módon a közeljövőben beépíteni a programba, ezáltal tovább fejlesztve és gyakorlással elmélyítve a tudásomat.

Véleményem szerint ebből a programból nagyon hasznos eszköz válhat azok számára akik vezetni szeretnének tanulni és kell egy kis segítség a szabályok megértésében. Ha nem is váltja ki a KRESZ vizsga komplex rendszerét, legalább egy ahhoz közelítő eszköz lehet ami egy kicsit izgalmasabbá tudja tenni a szabályok tanulását. A közlekedési szabályok megértése és gyakorlatban való alkalmazása rengeteg figyelmet és gyakorlást igényel, és nagyon fontos, hogy mindenki aki résztvesz a közlekedésben megfelelően legyen készítve a szabályok használatára.

Irodalomjegyzék

- [1] WIKIPÉDIA: *Id Software*, https://en.wikipedia.org/wiki/Id_Software, Megtekintve: 2026.04.17.
- [2] OLEKSANDRA SOBCHYSHAK: *Unreal Engine*, <https://www.sciencedirect.com/science/article/pii/S0141938225003051>, Megtekintve: 2026.04.17.
- [3] UNREAL ENGINE: *Unreal Engine*, <https://www.unrealengine.com/community?lang=en-US>, Megtekintve: 2026.04.17.
- [4] GODOT ENGINE: *Godot (game engine)*, https://docs.godotengine.org/en/4.4/getting_started/introduction/first_look_at_the_editor.html, Megtekintve: 2026.04.17.
- [5] UNITY TECHNOLOGIES: *Unity Technologies*: <https://docs.unity3d.com/Manual/index.html>, Megtekintve: 2026.04.18.
- [6] UNITY DOCUMENTATION: *Introduction to components*: <https://docs.unity3d.com/Manual/Components.html>, Megtekintve: 2026.04.18.
- [7] UNITY DOCUMENTATION: *MonoBehaviour*: <https://docs.unity3d.com/ScriptReference/MonoBehaviour.html>, Megtekintve: 2026.04.18.
- [8] UNITY DOCUMENTATION: *NavMesh*: <https://docs.unity3d.com/ScriptReference/AI.NavMesh.html>, Megtekintve: 2026.04.22.
- [9] UNITY DOCUMENTATION: *Cinemachine*: <https://docs.unity3d.com/Packages/com.unity.cinemachine@3.1/manual/index.html>, Megtekintve: 2026.04.22.
- [10] WIKIPÉDIA: *KRESZ*: <https://hu.wikipedia.org/wiki/KRESZ>, Megtekintve: 2026.04.18.
- [11] KRESZ VIZSGA: *KRESZ*: <https://vizsgakozpont.hu/kozut/jarmuvezeto-kepzes>, Megtekintve: 2026.04.22.
- [12] JOGTÁR: *KRESZ táblák*: <https://net.jogtar.hu/jogszabaly?docid=97500001.kpm>, Megtekintve: 2026.04.22.

Ábrajegyzék

- [13] KRESZ VIZSGA: *Három pontos feladat*, <https://szuperjogsi.hu/images/slide/3/Bogi110.jpg>, 2026.04.20.
- [14] KRESZ TÁBLÁK: *Közlekedési táblák*, https://vezess2.p3k.hu/app/uploads/2007/02/2123_19519_1000x700.jpg, 2026.04.20.
- [15] KRESZ VIZSGA: *Három pontos feladat villamossal*, <https://www.beol.hu/helyi-eletstilus/2025/07/kresz-teszt-villamos-elsobbseg>, 2026.04.20.
- [16] KRESZ VIZSGA: *Három pontos feladat mentőautóval*, <https://www.feol.hu/helyi-kozelet/2024/12/kresz-teszt-mentoauto>, 2026.04.20.
- [17] KRESZ VIZSGA: *KRESZ feladat mentőautóval*, https://www.szon.hu/helyi-kozelet/2025/07/megelozhető-a-mento-a-legtobben-bebukjak-ezt-a-kerdest#google_vignette, 2026.04.20.
- [18] KRESZ VIZSGA: *Három pontos feladat körforgalommal*, <https://www.beol.hu/helyi-eletstilus/2026/03/kresz-teszt-korforgalom>, 2026.04.20.
- [19] DOOM: *DOOM képernyőkép*, https://upload.wikimedia.org/wikipedia/en/thumb/d/de/Doom_ingame_1.png/250px-Doom_ingame_1.png, 2026.04.20.
- [20] UNREAL ENGINE: *Unreal Engine kezelőfelület*, <https://cdn2.unrealengine.com/ue5-modeling-mode-tech-blog-sandsculpting-2844x1559-1885809c9ebd.JPG>, 2026.04.20.
- [21] GODOT ENGINE: *Godot Engine kezelőfelület*, <https://upload.wikimedia.org/wikipedia/commons/e/e3/Godot3.4.png>, 2026.04.20.
- [22] UNITY ENGINE: *Unity kezelőfelület*, <https://unity.com/releases/2019-3/editor-tools>, 2026.04.20.
- [23] PLANTUML: *PlantUML ábra*, <https://plantuml.com>, 2026.04.20.

Nyilatkozat

Alulírott *Fodor Győző Benedek*, büntetőjogi felelősségem tudatában kijelentem, hogy az általam benyújtott, *KRESZ alapú tanulás segítő játék fejlesztése Unity keretrendszerbe* című szakdolgozat önálló szellemi termékem. Amennyiben mások munkáját felhasználtam, azokra megfelelően hivatkozom, beleértve a nyomtatott és az internetes forrásokat is.

Aláírással igazolom, hogy az elektronikusan feltöltött és a papíralapú szakdolgozatom formai és tartalmi szempontból mindenben megegyezik.

Eger, 2026. április 21.

Fodor Győző Benedek
aláírás